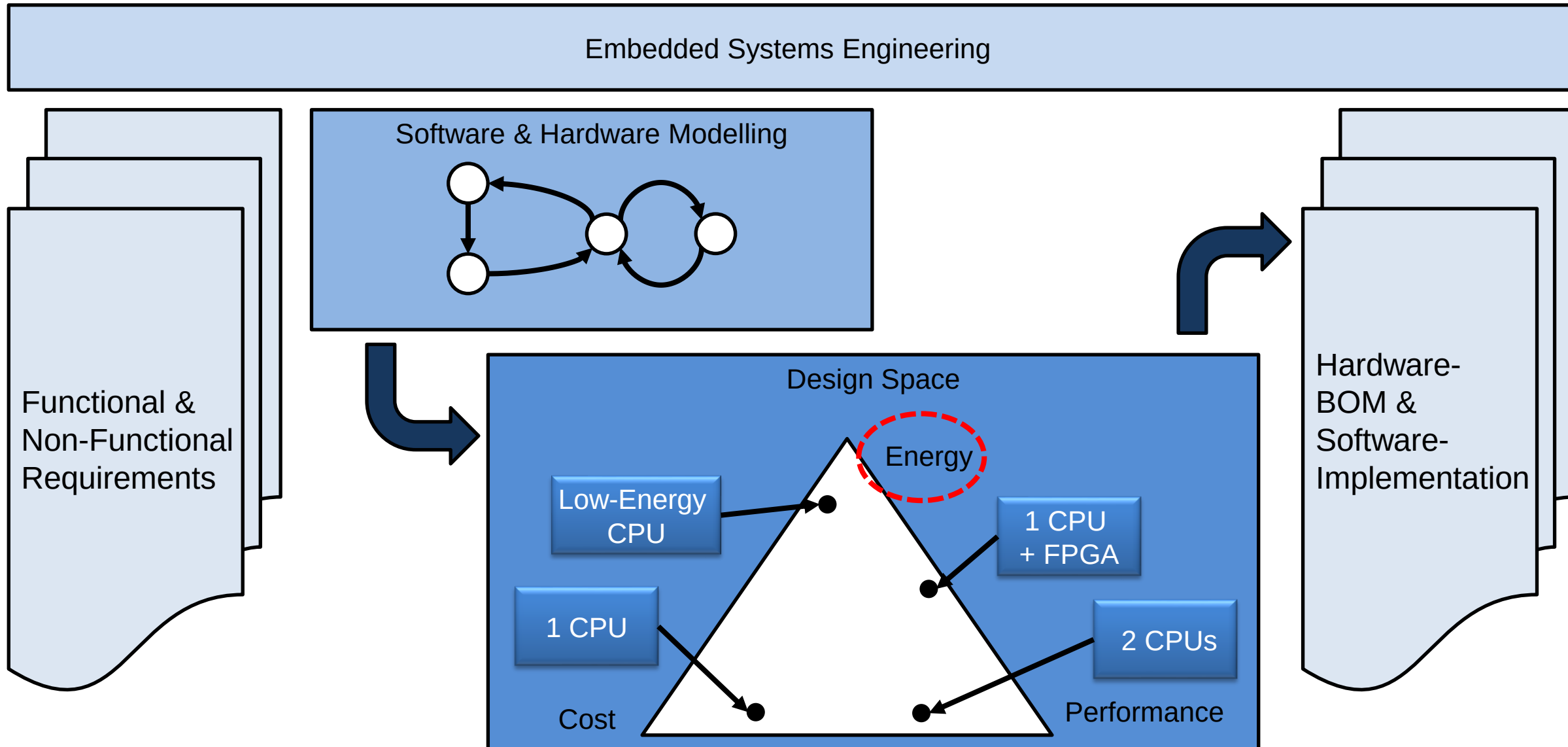


Embedded Software Engineering

Simon Künzli, Tobias Welti
HS 2025

Advance Organizer ESE



Learning Objectives

After today's lesson, the students ...

- **... know how to measure the overall energy consumption of an Embedded System**
- **... know how energy models support early design alternative evaluation**
- **... know Low Power Modes of different MCUs and can describe their functionality**
- **... know how these LPMs can lower the overall energy consumption**
- **... know how SW can influence the energy consumption**
- **... can determine relevant parameters of electronic components from data sheets**
- **... check whether a requirement concerning the energy consumption of an Embedded System is met**

Embedded System Design Considerations

- **What can have an impact on Energy Consumption?**

Low Power vs. Low Energy

- **Minimizing the *power consumption* ($P = V \cdot I$) is important for**
 - the design of the power supply (max. current that needs to be delivered)
 - PCB design to support power distribution
 - cooling (short term), which has an impact on cost and geometry

- **Minimizing *energy consumption* is important for**
 - battery dimension
 - Long-term temperature effects
 - High cost of energy (battery replacement, energy harvesting)

Embedded System Design Considerations

A few considerations on designing for low energy consumption:

- **What target values concerning energy consumption shall be achieved? We should know the answer before starting a new design**
- **Carefully study HW and SW used for your design. We can make use of low-power options only if we know about them**
- **Rapid prototyping (as known from the agile world) can be tricky as first energy consumption is usually ignored, and difficult to add at a later stage (similarity with secure design)**
- **You usually can't fix bad design decisions in HW by "magic" in SW**
- **In general, there is a tradeoff between low energy consumption and development time**

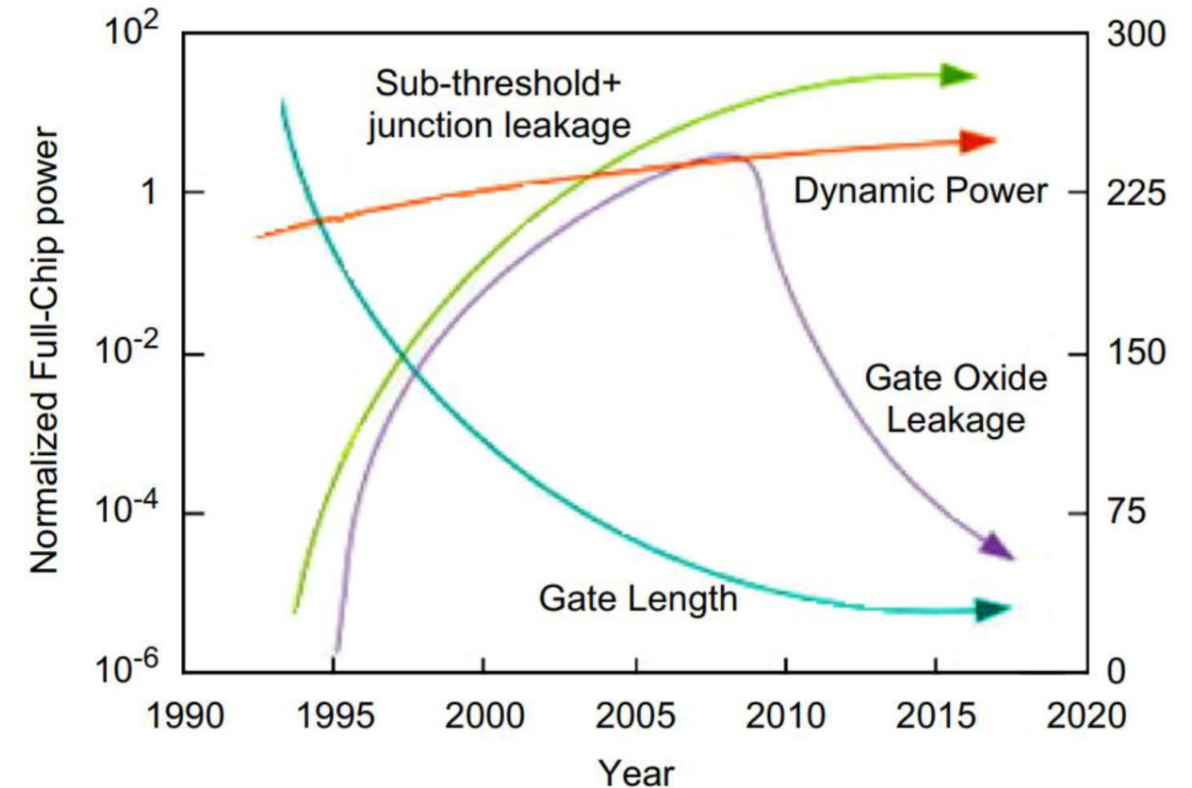
Static Power and Dynamic Power – Semiconductors

■ Static Power

- Leakage currents (gate-oxide/sub-threshold/junction) in transistors etc.
- more and more important (see figure)

■ Dynamic Power

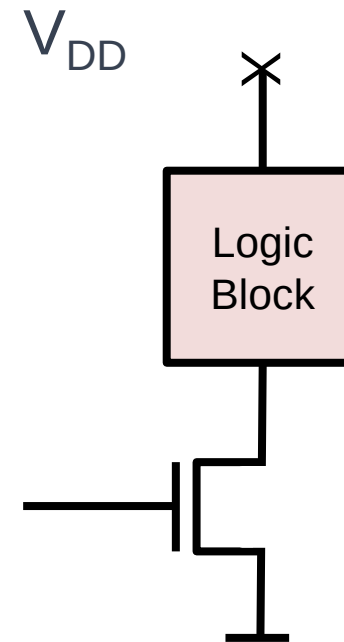
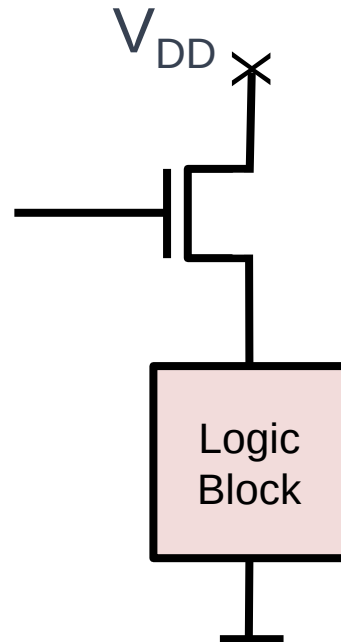
- Charging and discharging capacitors (Switching currents)
- Glitches (Short circuit paths between supply rails during switching)



[J. Xue, T. Li, Y. Deng, Z. Yu, Full-chip leakage analysis for 65 nm CMOS technology and beyond, Integration VLSI J. 43 (4) (2010) 353–364]

Reducing Static and Dynamic Power

- **Static Power (Power consumed when powered-up)**
 - Cut-off power supply to inactive units by power gating technique



Reducing Static and Dynamic Power

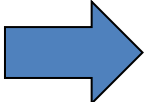
■ Dynamic Power (Power consumed during operation)

$$P = \alpha * C * V^2 * F$$

- Where α is the switching activity, C is the capacitive load, V is the supply voltage, and F is the frequency

→ To reduce dynamic power consumption, we must find ways to lower these parameters

Abstraction Levels



Level	Hardware	Software
System	Components and Sub-systems	Tasks that describe the system functionality
Module	Processors, MCUs, ASICs, Buses, Peripherals, Memory, Storage	Interacting SW modules
Block	Counter, Registers, ALUs	Program sequences, loops
Logic	Logic gates, Flip-Flops	Instructions (e.g. ADDS R1, R2, R3)
Device	Electronic components, transistors	Machine Code (e.g. 0x21ff)

Impact of System Components

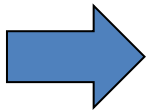
- **Get an overview of Components of System Components**
- **Example Traffic Light: Which component uses most of the energy?**

- **System**
 - Traffic Light Modules
 - Central Control Unit
 - Push Buttons for Pedestrians
 - Automatic Car Detection System
 - Internet Connection Box

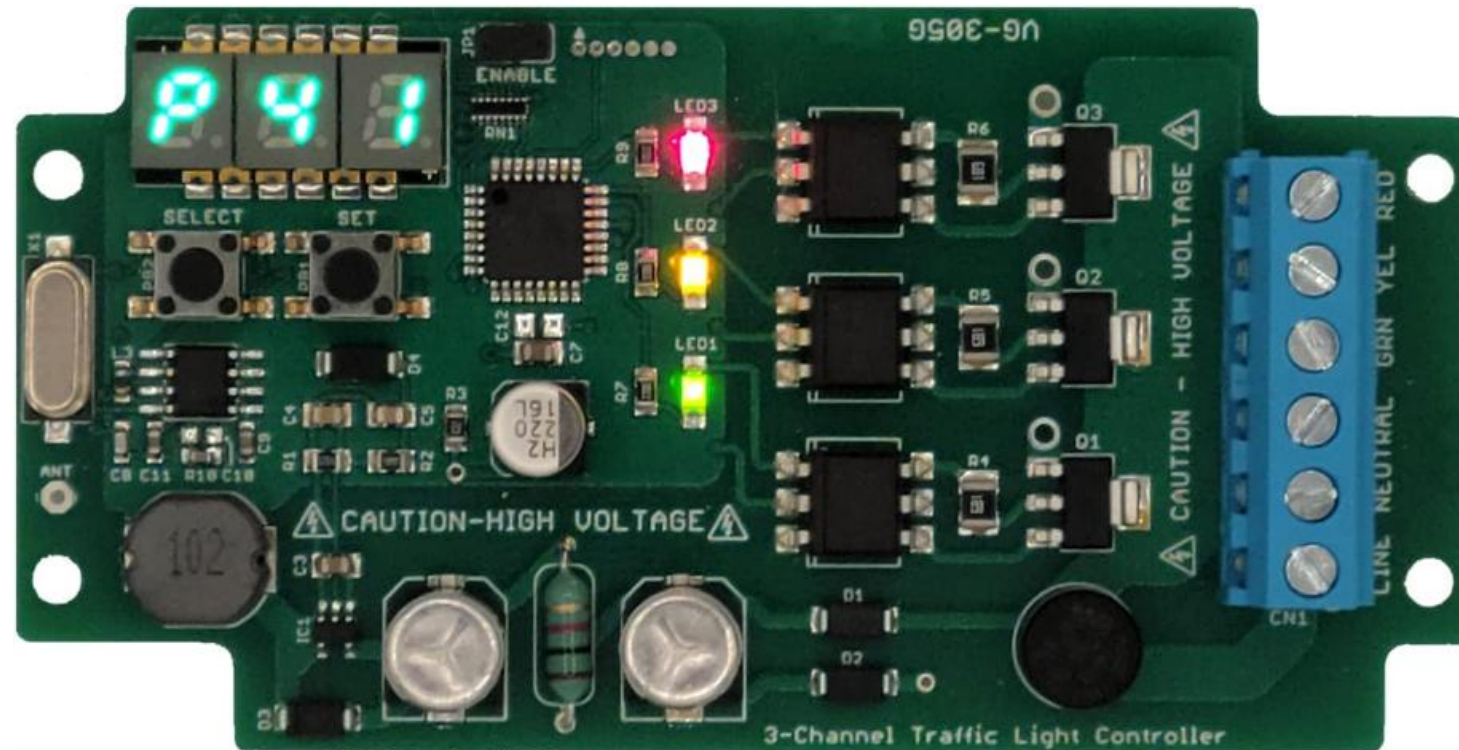
- **Where should we invest time for optimization?**

Abstraction Levels

Level	Hardware	Software
System	Components and Sub-systems	Tasks that describe the system functionality
Module	Processors, MCUs, ASICs, Buses, Peripherals, Memory, Storage	Interacting SW modules
Block	Counter, Registers, ALUs	Program sequences, loops
Logic	Logic gates, Flip-Flops	Instructions (e.g. ADDS R1, R2, R3)
Device	Electronic components, transistors	Machine Code (e.g. 0x21ff)



Impact of Modules



- **Example TI MSP430F5631**
 - Check on dynamic power consumption

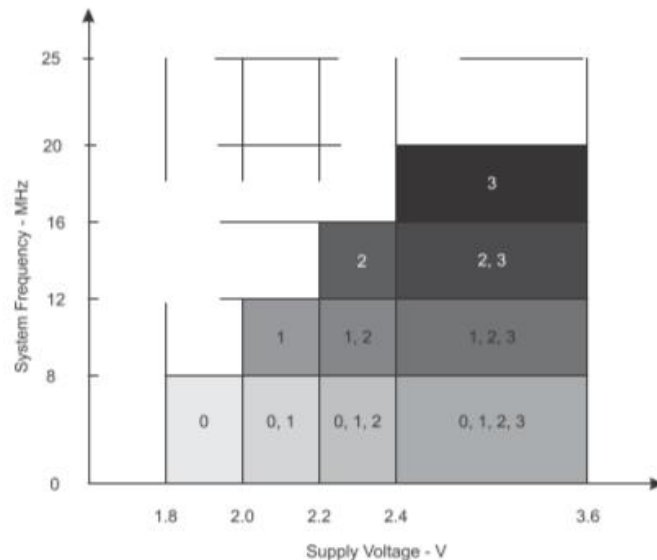


Figure 8-1. Frequency vs Supply Voltage

PARAMETER	EXECUTION MEMORY	V _{CC}	PMMCOREVx	FREQUENCY (f _{DCO} = f _{MCLK} = f _{SMCLK})								UNIT
				1 MHz		8 MHz		12 MHz		20 MHz		
				TYP	MAX	TYP	MAX	TYP	MAX	TYP	MAX	
I _{AM, Flash}	Flash	3 V	0	0.32	0.36	2.1	2.4					mA
			1	0.36		2.4		3.6	4.0			
			2	0.37		2.5		3.8				
			3	0.39		2.7		4.0		6.6		
I _{AM, RAM}	RAM	3 V	0	0.18	0.21	1.0	1.2					mA
			1	0.20		1.2		1.7	1.9			
			2	0.22		1.3		2.0				
			3	0.23		1.4		2.1		3.6		

MCU

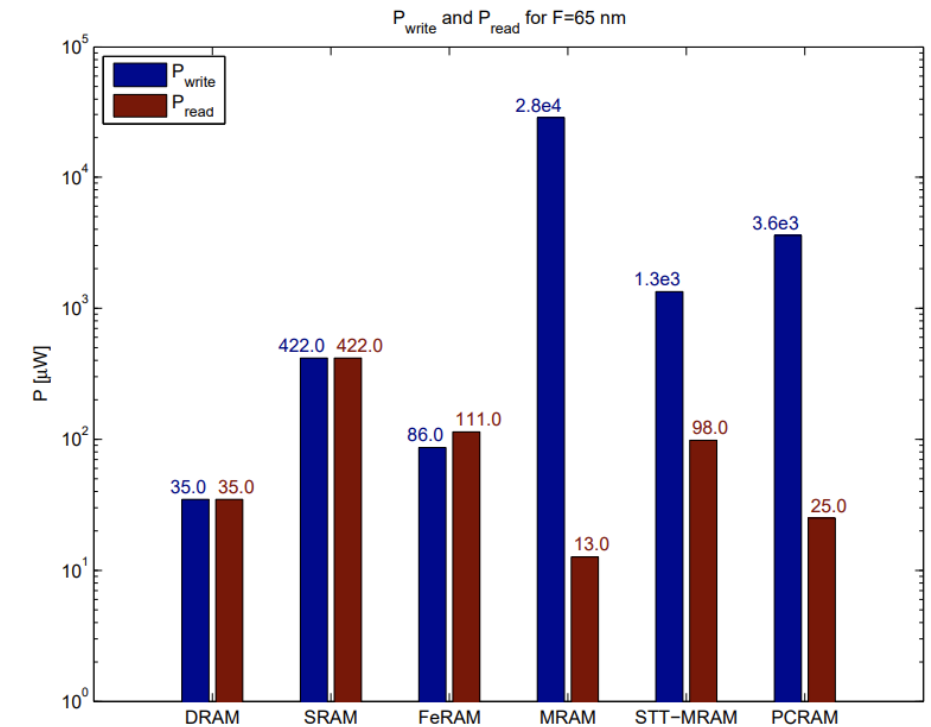
- **Example TI MSP430F5631**
 - Impact of operating temperature range

PARAMETER	V _{CC}	PMMCOREV _x	TEMPERATURE (T _A)								UNIT
			−40°C		25°C		60°C		85°C		
			TYP	MAX	TYP	MAX	TYP	MAX	TYP	MAX	
I _{LPM0,1MHz} Low-power mode 0 ^{(3) (9)}	2.2 V	0	71		75	87	81		85	99	μA
	3 V	3	78		83	98	89		94	108	
I _{LPM2} Low-power mode 2 ^{(4) (9)}	2.2 V	0	6.3		6.7	9.9	9.0		11	16	μA
	3 V	3	6.6		7.0	11	10		12	18	
I _{LPM3,XT1LF} Low-power mode 3, crystal mode ^{(5) (9)}	2.2 V	0	1.6		1.8	2.4	4.7		6.5	10.5	μA
		1	1.6		1.9		4.8		6.6		
		2	1.7		2.0		4.9		6.7		
	3 V	0	1.9		2.1	2.7	5.0		6.8	10.8	
		1	1.9		2.1		5.1		7.0		
		2	2.0		2.2		5.2		7.1		
		3	2.0		2.2	2.9	5.4		7.3	12.6	

RAM-Technologies

■ Comparison of different RAM technologies

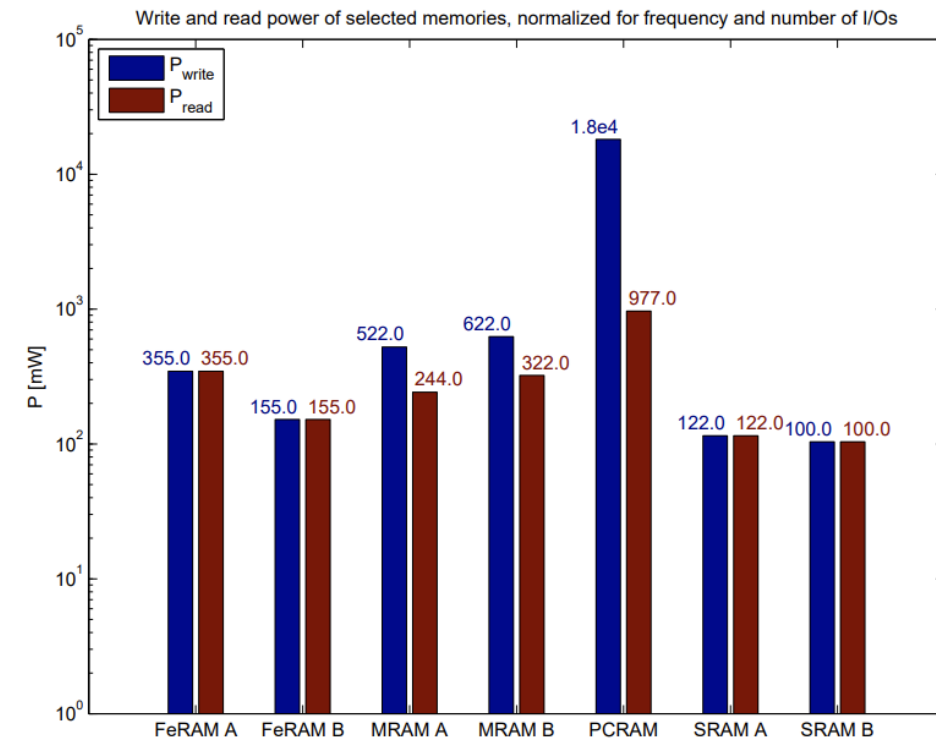
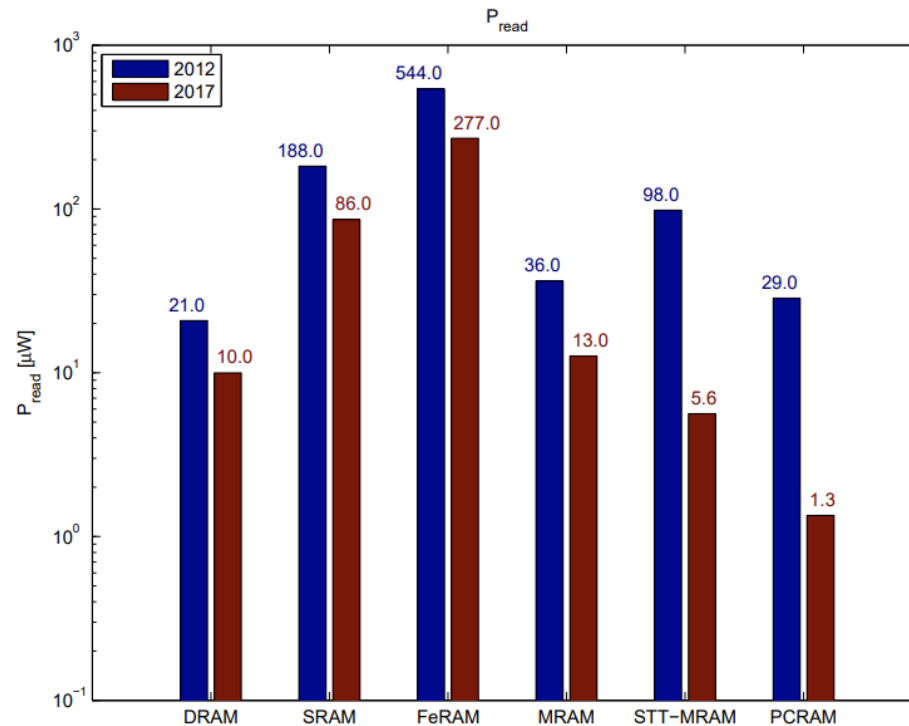
	Storage mechanism	Cell elements	Non-volatility
DRAM	Charge on a capacitor	1T1C	Volatile
SRAM	Inter locked state of logic gates	6T	Volatile
FeRAM	Remnant polarization on a ferroelectric capacitor	1T1C	Non-volatile
MRAM	Magnetization of ferromagnetic layer	1T1R	Non-volatile
STT-MRAM	Magnetization of ferromagnetic layer	1T1R	Non-volatile
PCRAM	Reversibly changing amorphous and crystalline phases	1T1R	Non-volatile



Source: Magnus Moreau, "Estimating the Energy Consumption of Emerging Random Access Memory Technologies", Master's Thesis, NTNU, Trondheim, 2013

RAM-Technologies

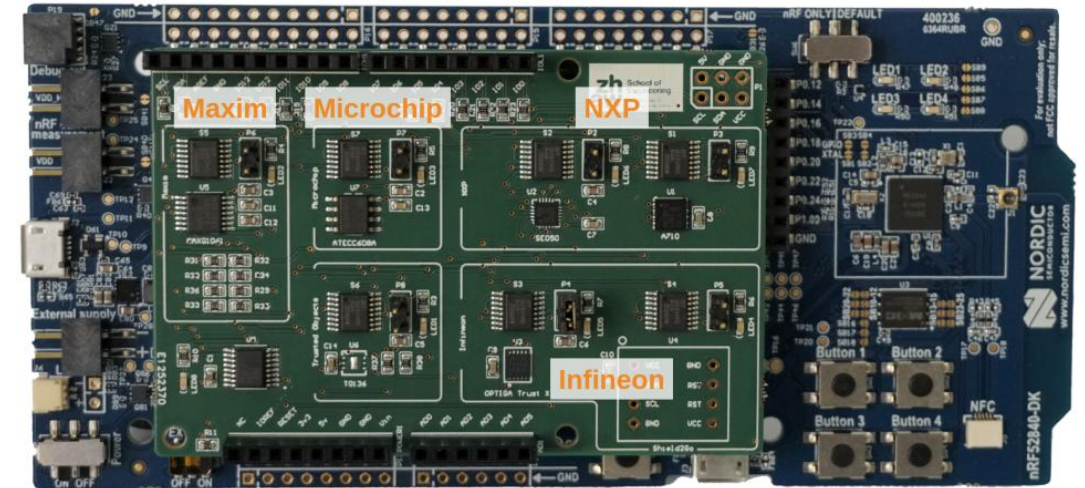
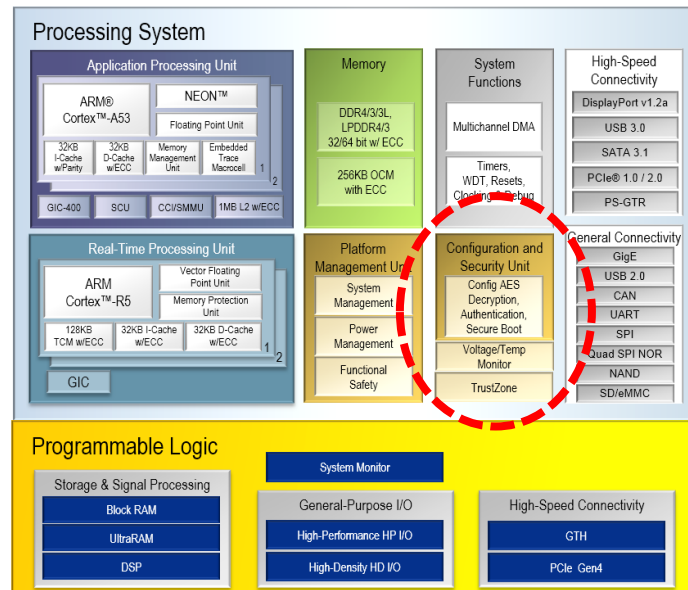
- Evolution of power consumption over the years
- Measurements comparing different manufacturers per same technology



Source: Magnus Moreau, "Estimating the Energy Consumption of Emerging Random Access Memory Technologies", Master's Thesis, NTNU, Trondheim, 2013

Secure Elements

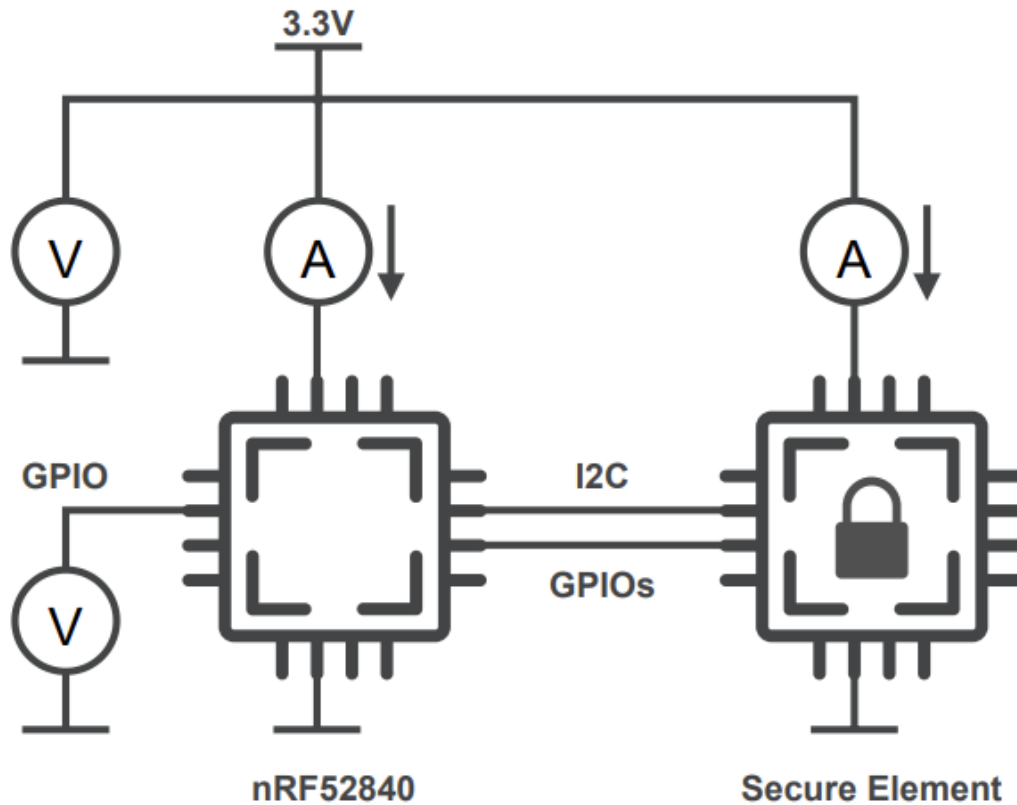
- A secure element is an integrated circuit (IC) for executing cryptographic algorithms (mostly) in hardware
- Similar to the Configuration and Security Unit on the Ultra96:



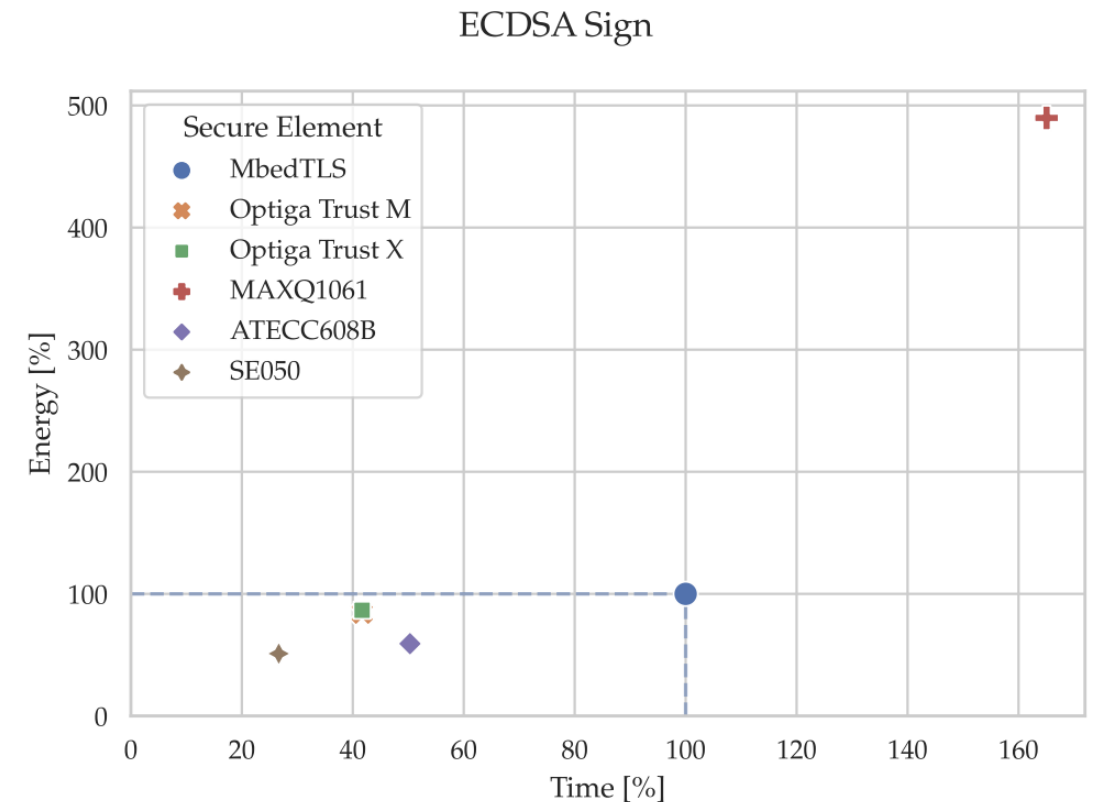
- Power Measurements have been performed at InES to compare several Secure Elements by various manufacturers

Secure Elements

■ Measurement Setup

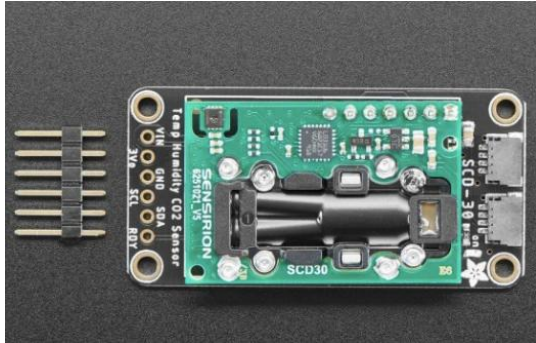


■ Comparison for ECDSA Signing Operation

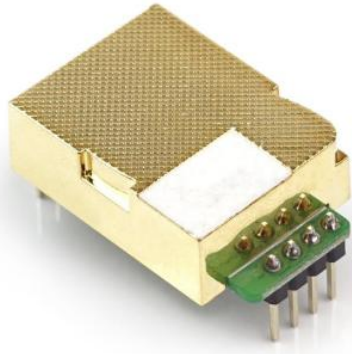


Source: Nosedá, M.; Zimmerli, L.; Schläpfer, T.; Rüst, A. Performance Analysis of Secure Elements for IoT. IoT 2022, 3, 1–28. <https://doi.org/10.3390/iot3010001>

CO2 Sensors



- **Non-Dispersive Infrared (NDIR) Sensors**



- **Metal Oxide Sensors (MOX)**



- **Photoacoustic Spectroscopy (PAS)**

CO2 Sensors

- Photoacoustic Spectroscopy
- Example: XENSIV™ PAS CO2 by Infineon



4.1.5 The current rating and power consumption

All parameters specified in the following sections refer to the operating conditions unless otherwise specified:
VDD3.3 = 3.3V, VDD12 = 12V, T_{amb} = 25°C, % r.H. = 30 %, p = 1013 hPa.

Table 7 Current rating

Parameter	Symbol	Pin	Values			Unit	Note or Test Condition
			Min.	Typ.	Max.		
Peak current ¹⁾	I _{peak 12}	VDD12		130	150	mA	
Peak current ¹⁾	I _{peak 3.3}	VDD3.3		10		mA	
Average current ¹⁾	I _{avg 12}	VDD12		0.8		mA	
Average current ¹⁾	I _{avg 3.3}	VDD3.3		6.1		mA	
Average power ¹⁾	P _{avg}			30		mW	

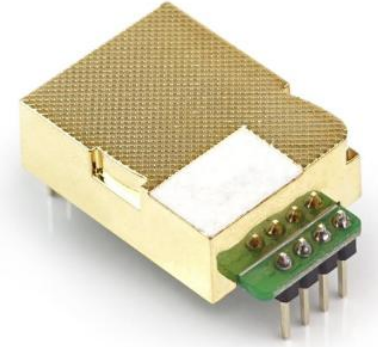
Power consumption can be optimized further. For more details please refer to our application note section at the product [web page](#).

Features

- **Operating range:** 0 ppm to 32000 ppm
- **Accuracy:** ± (30 ppm +3%) of reading between 400 ppm and 5000 ppm
- **Lifetime:** 10 years
- **Interface:** I2C, UART, and PWM
- **Package dimension:** 13.8 x 14 x 7.5 mm³
- **RoHS compliant**

CO2 Sensors

- Metal Oxide Sensor
- Example: MH-Z19C by Winsen Electronics



Main parameters

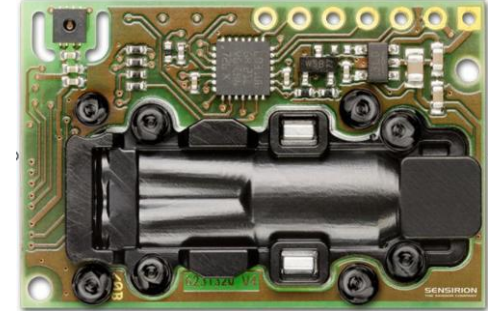
Model No.	MH-Z19C
Detection Gas	CO2
Working voltage	5.0 ± 0.1V DC
Average current	< 40mA (@5V power supply)
Peak current	125mA (@5V power supply)
Interface level	3.3 V (Compatible with 5V)
Detection Range	400~5000ppm(optional)
Output signal	Serial Port (UART) (TTL level 3.3V)
	PWM
Preheat time	1 min
Response Time	T ₉₀ < 120 s
Working temperature	-10 ~ 50 °C
Working humidity	0 ~ 95% RH (No condensation)
Weight	5 g
Lifespan	> 5 years

<200mW on average

Heating? Waiting time?

CO₂ Sensors

- Non-Dispersive Infrared (NDIR) Sensors
- Example: SCD30 NDIR CO₂ Sensor by Sensirion



Electrical Specifications

Parameter	Conditions	Value
Average current ¹⁰	Update interval 2 s	19 mA
Max. current	During measurement	75 mA
DC supply voltage (V _{dd_min} - V _{dd_max})	Min. and max. criteria to operate SCD30	3.3 V – 5.5 V
Interface	-	UART (Modbus Point to Point; TTL Logic), PWM and I ² C
Input high level voltage (V _{IH}) I ² C	Min. and max. criteria to operate SCD30	1.75 V - 3.0 V
Input high level voltage (V _{IH}) Modbus	Min. and max. criteria to operate SCD30	1.75 V – 5.5 V
Input low level voltage (V _{IL}) I ² C/Modbus	Min. and max. criteria to operate SCD30	- 0.3 V – 0.9 V
Output low level voltage (V _{OL}) I ² C/Modbus	I _{IO} = +8 mA, Max. criteria	0.4 V
Output high level voltage (V _{OH}) I ² C/Modbus	I _{IO} = -6 mA, Min. criteria	2.4 V

Table 4 SCD30 electrical specifications

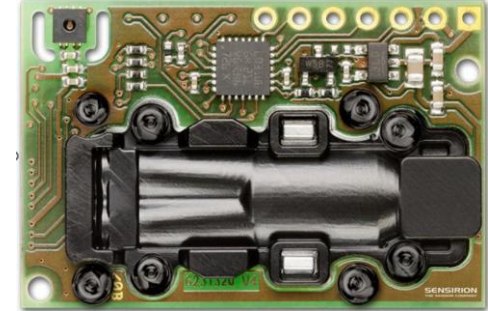
SENSOR MODULE SPECIFICATIONS

CO₂ Sensor Specifications

CO ₂ measurement range	0 – 10'000 ppm
Accuracy	± 30 ppm ± 3% (25 °C, 400 – 10'000 ppm)
Repeatability	10 ppm
Temperature stability	2.5 ppm / °C (0 – 50 °C)
Response time (t ₆₃)	25 s

CO2 Sensors

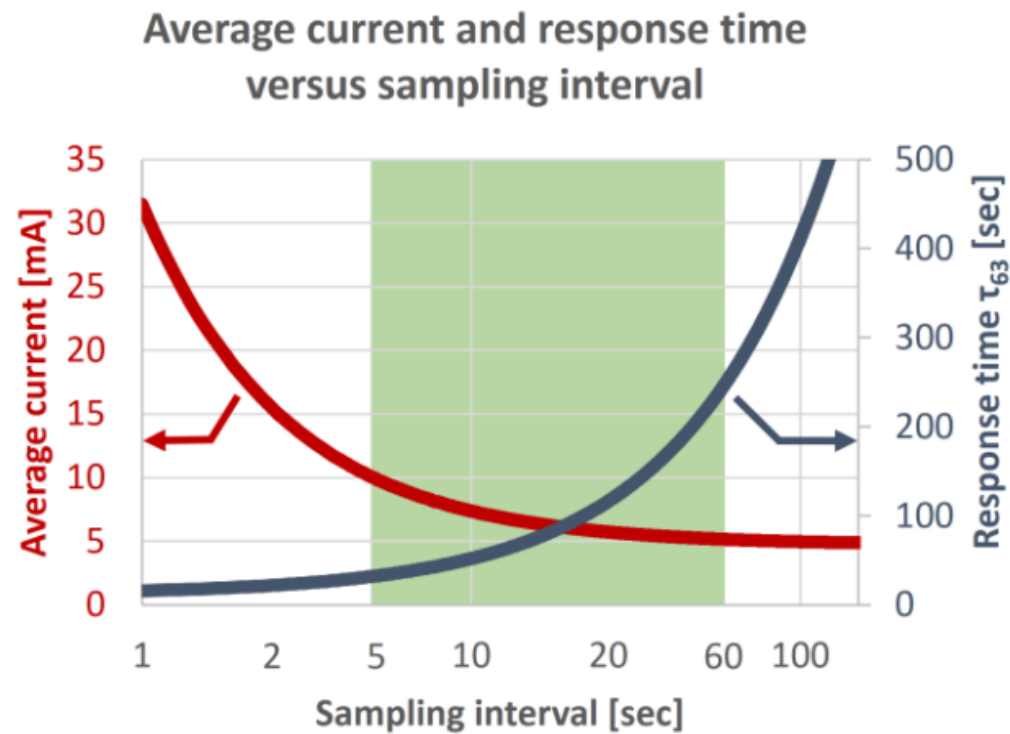
- Non-Dispersive Infrared (NDIR) Sensors
- Example: SCD30 NDIR CO2 Sensor by Sensirion



Electrical Specifications

Parameter	Condition
Average current ¹⁰	Update in
Max. current	During m
DC supply voltage ($V_{dd_{min}}$ - $V_{dd_{max}}$)	Min. and
Interface	-
Input high level voltage (V_{IH}) I2C	Min. and
Input high level voltage (V_{IH}) Modbus	Min. and
Input low level voltage (V_{IL}) I2C/Modbus	Min. and
Output low level voltage (V_{OL}) I2C/Modbus	$I_{IO} = +8 \text{ mA}$
Output high level voltage (V_{OH}) I2C/Modbus	$I_{IO} = -6 \text{ mA}$

Table 4 SCD30 electrical specifications



MEASUREMENT SPECIFICATIONS

Measurement Specifications

Measurement range	0 – 10'000 ppm
Accuracy	$\pm 30 \text{ ppm} \pm 3\%$ (25 °C, 400 – 10'000 ppm)
Resolution	10 ppm
Temperature compensation	2.5 ppm / °C (0 – 50 °C)
Measurement cycle	25 s

Impact of Energy Source

The energy source influences HW and SW design:

■ Batteries

- Supply depending on the battery technology
- Several factors might impact battery life: age, energy level, ***consumption patterns***

■ Energy Harvesting

- Only «run» ES when enough energy is available
- Special precautions needed in SW to have a clean re-start

■ Mains-Powered Devices

- Low energy consumption might still be beneficial, e.g. to keep temperature low

Software Options

■ Task Scheduling

- Optimize your software's task scheduling to minimize the time that the CPU is active. Use sleep modes or idle states when possible → See following slides

■ Data Transmission Optimization

- Use power-efficient transmission protocols and optimize the amount of data sent.
- Use efficient data compression or send data in burst transmissions instead of sending it continuously.

■ Sensor Fusion

- Combine data from multiple sensors to reduce the number of times the CPU wakes up

■ Energy-efficient algorithms

- Optimize your algorithms to minimize computational and memory requirements.
- Use efficient data structures, minimize unnecessary data transfers, and reduce computational load

■ Low-power software design

- Reduce unnecessary polling, minimizing busy-wait loops, and using efficient interrupt-driven programming to respond to events.

Software Options

■ Advanced Compiler Optimization

Table 3 Selected C/C++ programs and their energy consumption (CPU and memory) in Joules by optimization level

Level Program	-O0	-O1	-O2	-O3	-Ofast	-Os
MMC	354.934	237.568	239.599	240.192	238.445	227.809
Grades	352.228	257.912	246.786	248.958	245.701	249.664
Bzip	406.914	214.438	216.539	215.882	213.496	234.162
Bzip2	402.516	212.324	212.327	215.639	213.947	229.156
Oggenc	523.992	216.264	195.844	190.472	180.038	213.51
Pbrt	556.222	141.461	142.815	136.84	134.378	209.164

Source: Branco, David & Rangel Henriques, Pedro. (2016). Impact of GCC Optimization Levels in Energy Consumption During Program Execution. Acta Electrotechnica et Informatica. 16. 20-26. 10.15546/aei-2016-0004.

Software - Dynamic Power Consumption

- **Energy Saving Potential during operation**
 - Dynamic Voltage and Frequency Scaling techniques
 - MCU Low Power Modes
 - Peripherals Management, Clock Gating

- **Dynamic voltage scaling (DVS):**
 - Very effective in reducing power, since lowering the voltage has a squared effect on active power consumption.

- **Dynamic frequency scaling (DFS):**
 - Remember $P = a \cdot C \cdot V^2 \cdot F$, DFS also known as CPU throttling

MCU: Low-Power Modes

■ Example TI MSP430F5xxx

MSP430F5638, MSP430F5637, MSP430F5636, MSP430F5635
MSP430F5634, MSP430F5633, MSP430F5632, MSP430F5631, MSP430F5630
SLAS650G – MAY 2010 – REVISED SEPTEMBER 2020
www.ti.com

9.4 Operating Modes

These devices have one active mode and seven software-selectable low-power modes of operation. An interrupt event can wake up the device from any of the low-power modes, service the request, and restore back to the low-power mode on return from the interrupt program.

Software can configure the following operating modes:

- Active mode (AM)
 - All clocks are active
- Low-power mode 0 (LPM0)
 - CPU is disabled
 - ACLK and SMCLK remain active, MCLK is disabled
 - FLL loop control remains active
- Low-power mode 1 (LPM1)
 - CPU is disabled
 - FLL loop control is disabled
 - ACLK and SMCLK remain active, MCLK is disabled
- Low-power mode 2 (LPM2)
 - CPU is disabled
 - MCLK, FLL loop control, and DCOCLK are disabled
 - DC generator of the DCO remains enabled
 - ACLK remains active
- Low-power mode 3 (LPM3)
 - CPU is disabled
 - MCLK, FLL loop control, and DCOCLK are disabled
 - DC generator of the DCO is disabled
 - ACLK remains active
- Low-power mode 4 (LPM4)
 - CPU is disabled
 - ACLK is disabled
 - MCLK, FLL loop control, and DCOCLK are disabled
 - DC generator of the DCO is disabled
 - Crystal oscillator is stopped
 - Complete data retention
- Low-power mode 3.5 (LPM3.5)
 - Internal regulator disabled
 - No data retention
 - RTC enabled and clocked by low-frequency oscillator
 - Wake-up signal from RST/NMI, RTC_B, P1, P2, P3, and P4
- Low-power mode 4.5 (LPM4.5)
 - Internal regulator disabled
 - No data retention
 - Wake-up signal from RST/NMI, P1, P2, P3, and P4

8.5 Low-Power Mode Supply Currents (Into V_{CC}) Excluding External Current

over recommended ranges of supply voltage and operating free-air temperature (unless otherwise noted)^{(1) (2)}

PARAMETER	V _{CC}	PMMCOREVx	TEMPERATURE (T _A)								UNIT
			–40°C		25°C		60°C		85°C		
			TYP	MAX	TYP	MAX	TYP	MAX	TYP	MAX	
I _{LPM0,1MHz} Low-power mode 0 ^{(3) (9)}	2.2 V	0	71		75	87	81		85	99	μA
	3 V	3	78		83	98	89		94	108	
I _{LPM2} Low-power mode 2 ^{(4) (9)}	2.2 V	0	6.3		6.7	9.9	9.0		11	16	μA
	3 V	3	6.6		7.0	11	10		12	18	
I _{LPM3,XT1LF} Low-power mode 3, crystal mode ^{(5) (9)}	2.2 V	0	1.6		1.8	2.4	4.7		6.5	10.5	μA
		1	1.6		1.9		4.8		6.6		
		2	1.7		2.0		4.9		6.7		
	3 V	0	1.9		2.1	2.7	5.0		6.8	10.8	
		1	1.9		2.1		5.1		7.0		
		2	2.0		2.2		5.2		7.1		
		3	2.0		2.2	2.9	5.4		7.3	12.6	
I _{LPM3,VLO,WDT} Low-power mode 3, VLO mode, Watchdog enabled ^{(6) (9)}	3 V	0	0.9		1.2	1.9	4.0		5.9	10.3	μA
		1	0.9		1.2		4.1		6.0		
		2	1.0		1.3		4.2		6.1		
		3	1.0		1.3	2.2	4.3		6.3	11.3	
I _{LPM4} Low-power mode 4 ^{(7) (9)}	3 V	0	0.9		1.1	1.8	3.9		5.8	10	μA
		1	0.9		1.1		4.0		5.9		
		2	1.0		1.2		4.1		6.1		
		3	1.0		1.2	2.1	4.2		6.2	11	
I _{LPM3.5,RTC,VCC} Low-power mode 3.5 (LPM3.5) current with active RTC into primary supply pin DV _{CC} ⁽¹⁰⁾	3 V				0.5				0.8	1.4	μA

MCU: Low-Power Modes

■ Example STM32F429 (known from CT)

5.3 Low-power modes

By default, the microcontroller is in Run mode after a system or a power-on reset. In Run mode the CPU is clocked by HCLK and the program code is executed. Several low-power modes are available to save power when the CPU does not need to be kept running, for example when waiting for an external event. It is up to the user to select the mode that gives the best compromise between low-power consumption, short startup time and available wakeup sources.

The devices feature three low-power modes:

- Sleep mode (Cortex[®]-M4 with FPU core stopped, peripherals kept running)
- Stop mode (all clocks are stopped)
- Standby mode (1.2 V domain powered off)

126/1751

RM0090 Rev 19



Entering and leaving low-power modes

Read the MCU manuals, ask your search engine for examples... or ChatGPT

```
// Example Code for TI MSP430
#include <msp430.h>

int main(void)
{
    [...]

    // Configure the low-power mode
    __bis_SR_register(LPM4_bits); // Enter Low-Power Mode 4 (LPM4)

    [...]
    return 0;
}

// Interrupt service routine for the Port 1 interrupt
__interrupt void Port1_ISR(void)
{
    // Clear the interrupt flag

    __bic_SR_register_on_exit(LPM4_bits); // Exit Low-Power Mode 4 (LPM4)
}
```

```
// Example Code for ARM-M0+
#include "cmsis_device.h"

int main(void)
{
    // Initialize the microcontroller and peripherals here

    // Enter Sleep mode
    __WFI(); // Wait for interrupt

    // After waking up, the code continues here

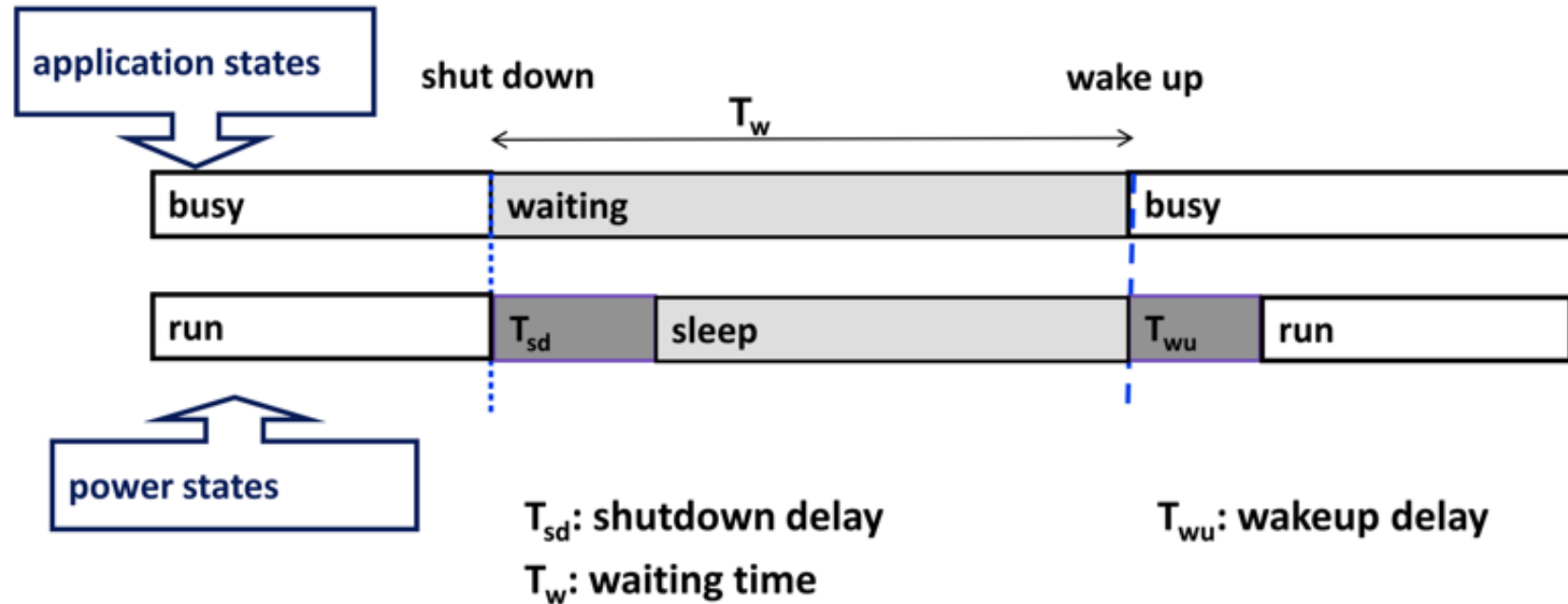
    while (1) {
        // Main application loop
    }
}

// Interrupt service routine for a wake-up event
void YourWakeupEvent_Handler(void)
{
    // Handle the wake-up event here

    // Exit Sleep mode (if needed)

    // If using __WFI() in main, the microcontroller will automatically
    // exit Sleep mode when an interrupt occurs.
}
```

Dynamic Power Management (DPM)



Source: L.Thiele: Lecture on Embedded Systems, ETHZ, 2018

E.g. in TI MSP430's LPM3.5, data will not be retained, hence it will have to be reloaded after leaving LPM (during T_{wu})

→ Additional code needed

→ Tradeoff between additional code vs. Energy savings. Is it worth the effort?

Summary – Impact on Energy Consumption

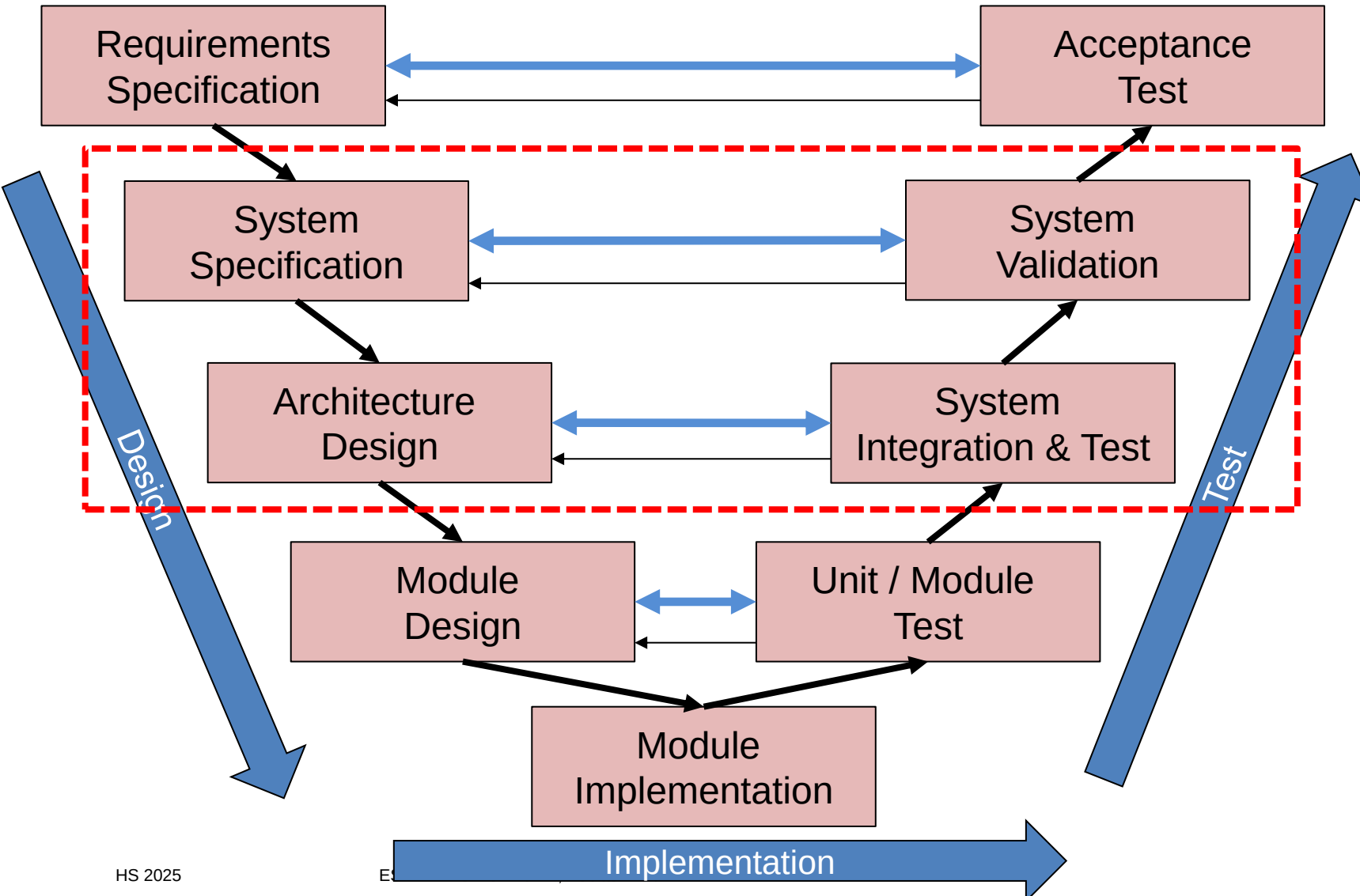
■ Hardware

- Electronics design
- Component Selection
- Power Supply

■ Software

- Control of Energy saving options
 - Low Power Modes of MCUs
 - Dynamic Power Management
- Configuration of Core Voltage and Clock Frequency
- Algorithm Implementation
- Peripherals Management (switch-off unused peripherals)

V – Model



- **Verification of Models already during design phases**

Energy Models

- To have early estimations of energy consumption, energy models are needed
- First approach based on datasheet (Example CO2 sensor):

4.1.5 The current rating and power consumption

All parameters specified in the following sections refer to the operating conditions unless otherwise specified:
VDD3.3 = 3.3V, VDD12 = 12V, $T_{amb} = 25^{\circ}\text{C}$, % r.H. = 30 %, p = 1013 hPa.

Table 7 Current rating

Parameter	Symbol	Pin	Values			Unit	Note or Test Condition
			Min.	Typ.	Max.		
Peak current ¹⁾	$I_{peak\ 12}$	VDD12		130	150	mA	
Peak current ¹⁾	$I_{peak\ 3.3}$	VDD3.3		10		mA	
Average current ¹⁾	$I_{avg\ 12}$	VDD12		0.8		mA	
Average current ¹⁾	$I_{avg\ 3.3}$	VDD3.3		6.1		mA	
Average power ¹⁾	P_{avg}						

Power consumption can be optimized further by the product [web page](#).



210.24 J
per meas.

Table 2 Power consumption split

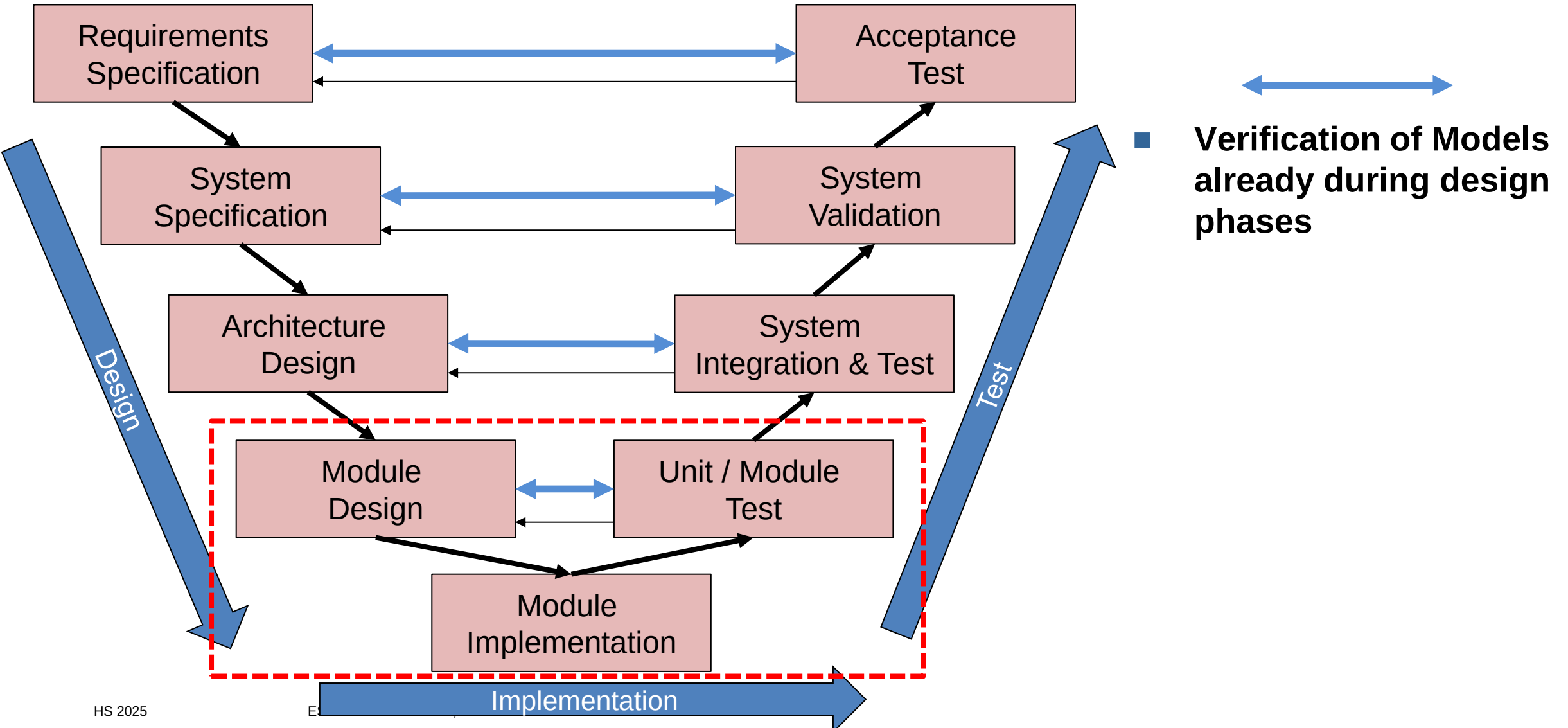
Parameter	Power up	During Measurement			Idle		Unit
	3.3 V	3.3 V	12 V		3.3 V	12 V	
Average current	9	8	70		6	0.5	mA
Average power	29.7	26.4	840		19.8	6	mW
Duration	0.8	0.7	0.2		-		s

Energy Models

- **Static analysis and energy models allow energy consumption prediction**
- **For Software: Specific Power Profiling Tools**
 - For microcontroller-based circuits, some development tools and software libraries offer power profiling features. These tools record the current consumption of the microcontroller and its peripherals over time, allowing you to analyze the energy consumption of your code.

→ use these results to refine your models
- **Finally, simple Excel sheets can be used for first estimations of the overall power consumption of your Embedded System**
→ **High-level verification of NFRs concerning Energy consumption**

V – Model



Energy Measurements

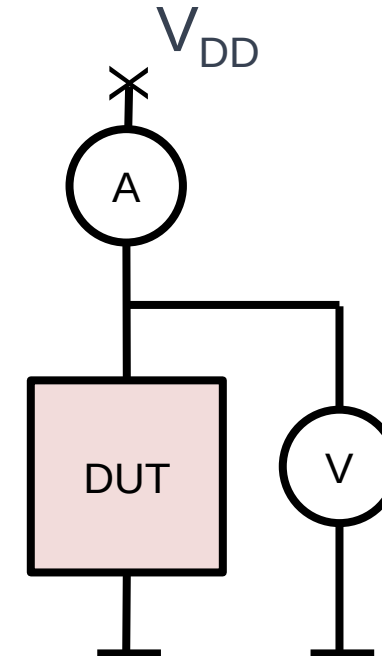
- **When measuring energy consumption, it's essential to consider the measurement accuracy, sampling rate, and measurement duration.**
- **Power consumption can vary depending on the operational state of the circuit (e.g., active, sleep, or standby modes)**
 - **We may need to capture power data under different scenarios to get a complete picture of energy usage**

Multimeter(s)



■ Current and Voltage Measurements:

- A simple method for estimating power consumption is to measure the current and voltage across the circuit using 2 multimeters.
- Power $P = V * I$

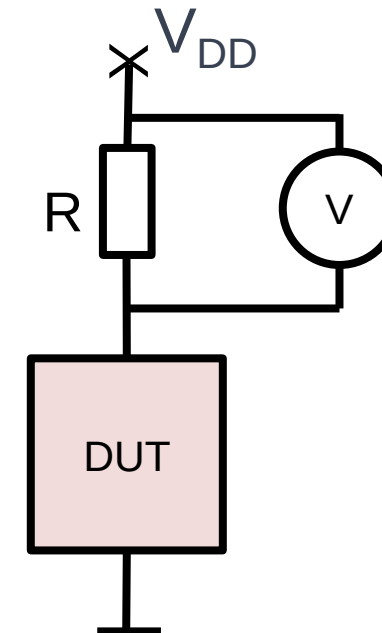


Multimeter(s)



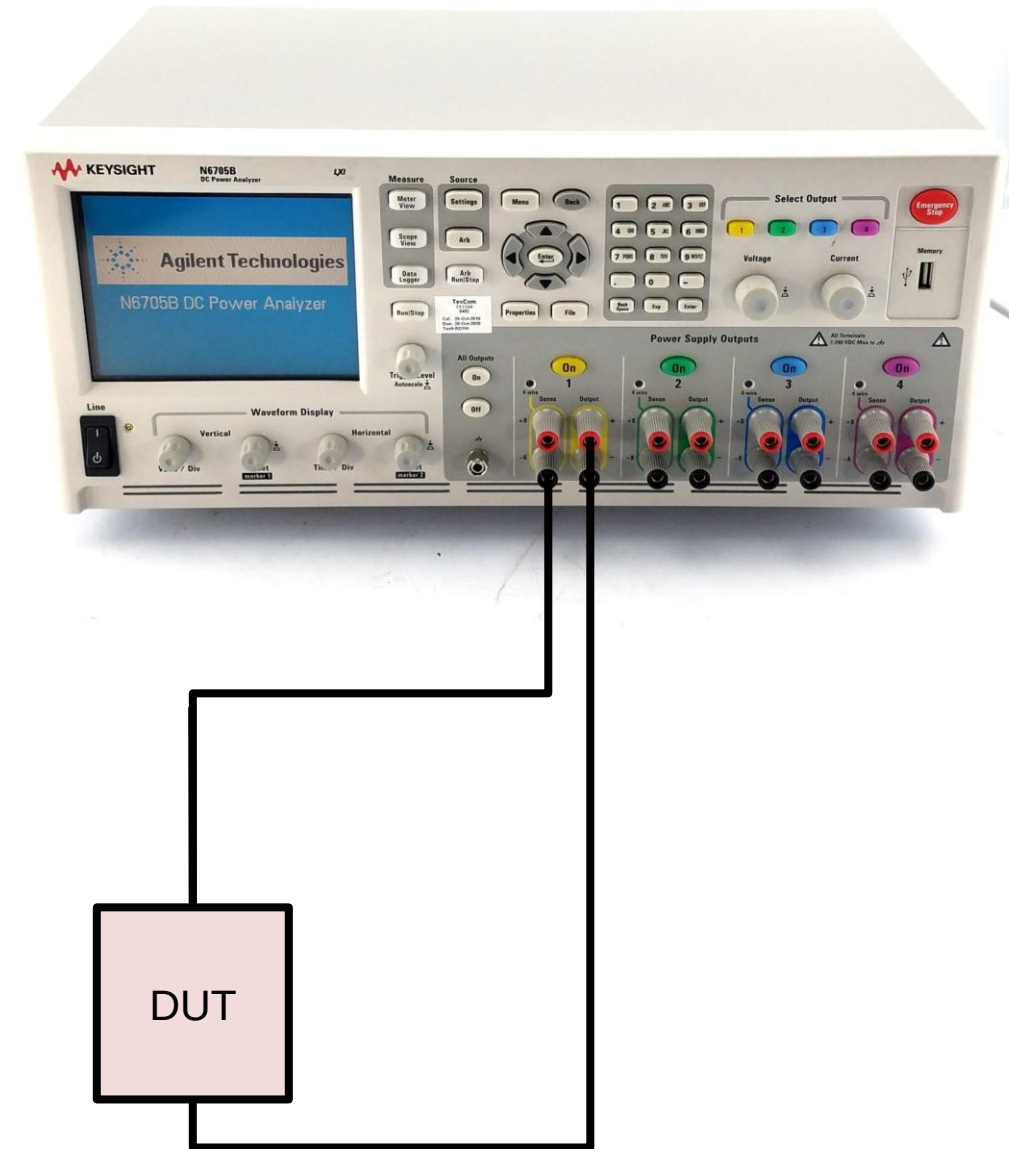
■ Current Shunt Resistors

- The voltage across the shunt resistor is proportional to the current flowing through it.
- $P = V/R * V_{DD}$



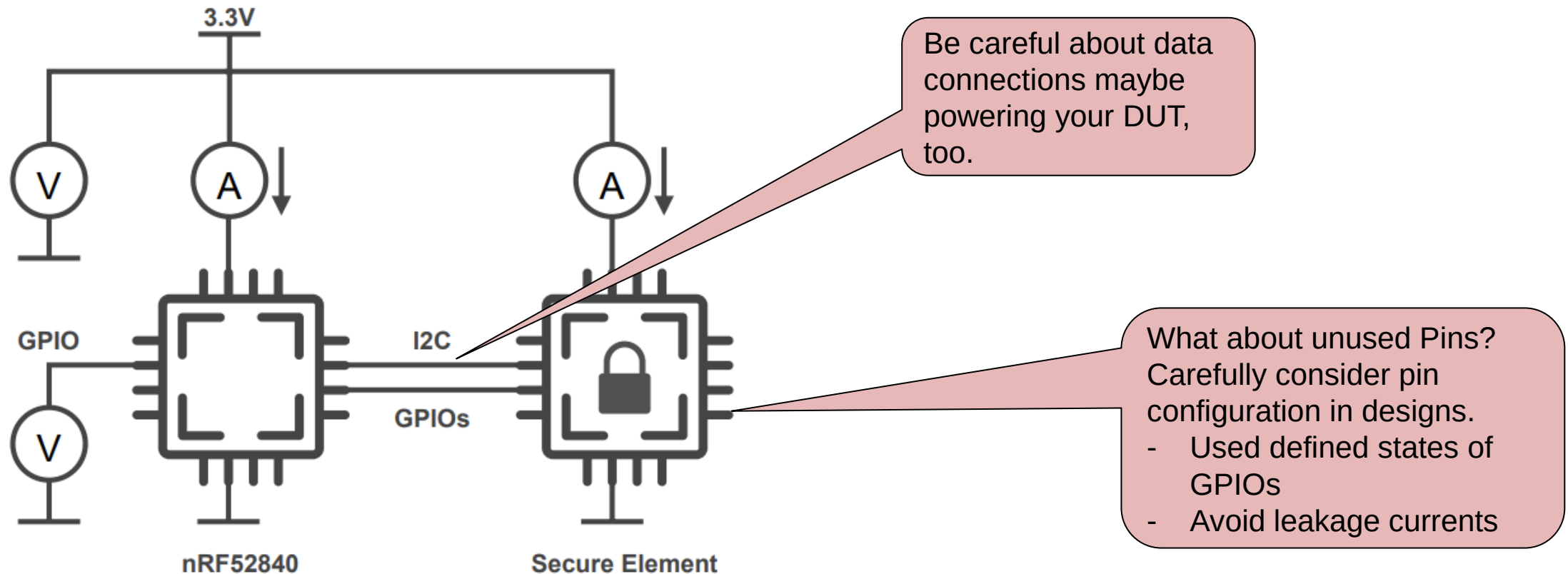
Power Analyzer

- **Power Analyzer used to measure the power consumption of a DUT using a single device**
 - Supplies at constant voltage
 - Provides «unlimited» current
- **2nd wire pair to compensate for the cables connecting to the DUT**



Energy Measurements

■ Remember the measurement setup for the Secure Elements experiments?



Source: Nosedá, M.; Zimmerli, L.; Schläpfer, T.; Rüst, A. Performance Analysis of Secure Elements for IoT. IoT 2022, 3, 1–28. <https://doi.org/10.3390/iot3010001>

Summary

- **Power consumption of a design is impacted by hardware and software**
- **Use energy models in early design phases (datasheets, energy profiling tools, static code analysis)**
- **Don't delay power consumption optimization to late project phases**
- **Static and dynamic power consumption must be considered**